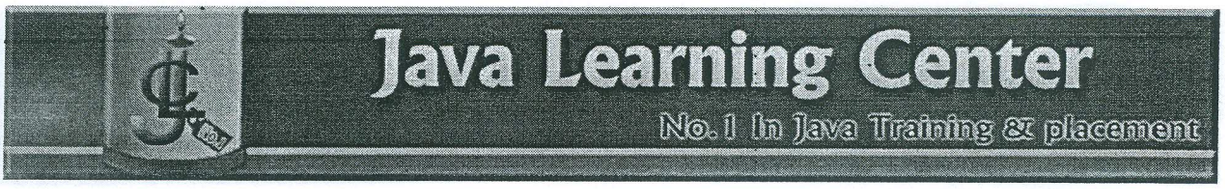
Java-22



Java Learning Center

java.io

Put into Knowledge

Doc Version - 1.0

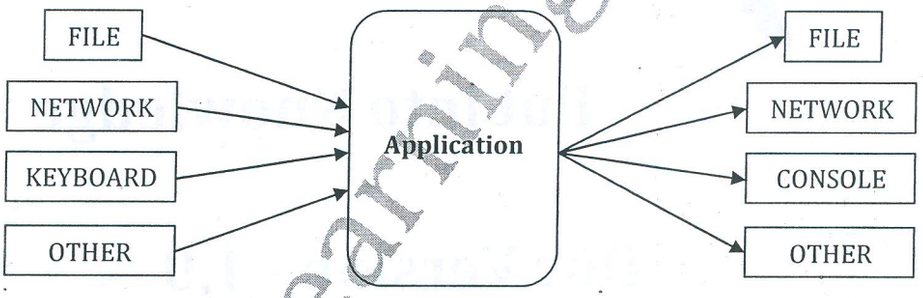
Author

Srinivas Dande

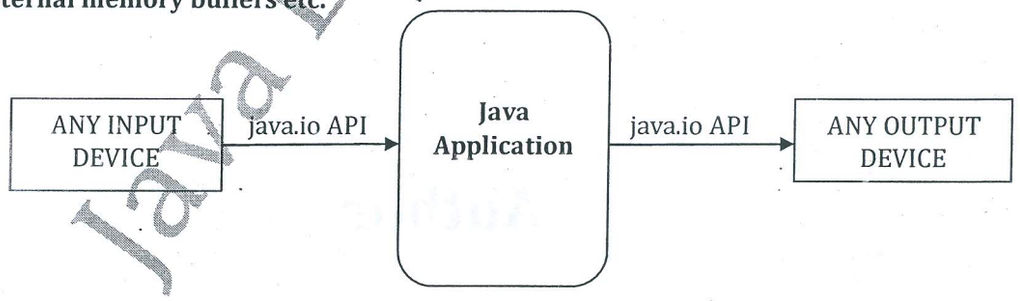
Kamlesh Kumar Singh

Introduction

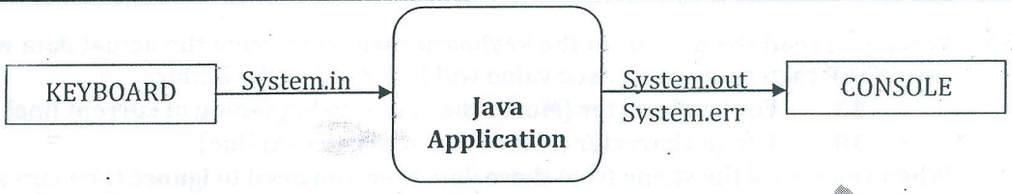
- When you develop any java application then you will get the requirement to interact with various input -output devices.
- Input-Output devices are the part of underlying hardware and operating system. If you want to interact with I/O devices from java program then you have to use some native implementations that may increase the maintenance of application when you change device or os.
- So to resolve this problem Java Vendor has given API in java.io package by which you can interact with various input -output devices from java program.
- Each classes represent on source or device.
- If you want to change the device then you need to create the object of another class, remaining implementaion will be same.
- All the methods and some constructors of java.io package are throwing checked exception called java.IOException.
- Because of checked exception you must have to report about the exception by writing try catch block or by using throws keyword.



- Java.io package provides classes for system input and output through files, network streams, internal memory buffers etc.



- Some input-output stream will be initialized by the JVM automatically at JVM startup and these streams are available in System class as in, out and err reference variable.



- **in** reference refers the default input device i.e. KEYBOARD.
- **out** and **err** will refer by default the default output device i.e. CONSOLE.

Lab1329.java

```
public class Lab1329 {
public static void main(String[] args) throws
Exception{
System.out.println("Enter 1st character");
int var = System.in.read();
System.out.println(var + "\t" + (char) var);
System.out.println("Enter 2nd Character ");
var = System.in.read();
System.out.println(var + "\t" + (char) var);
System.out.println("Enter 3rd Character ");
var = System.in.read();
System.out.println(var + "\t" + (char) var);
}
}
```

Lab1330.java

```
public class Lab1330 {
public static void main(String[] args) throws Exception {
System.out.println("Enter Data [ * -> Exit ]");
int count = 0;
while (true) {
int asc = System.in.read();
System.out.println(++count + "\t" + asc + "\t" + (char) asc);
if (asc == '*')break;
}
System.out.println("\n AFTER LOOP");
while (true) {
int asc = System.in.read();
System.out.println(asc + "\t" + (char) asc);
if (asc == 10) break;
} }}
```

Lab1331.java

```
import java.io.IOException;
public class Lab1331 {
public static void main(String[] args) throws
Exception {
JlcReader rd = new JlcReader();
System.out.println("Enter Sid");
String sid = rd.readLine();
System.out.println("Enter Name");
String nm = rd.readLine();
System.out.println("Enter Phone");
String phn = rd.readLine();
System.out.println("Enter Fee");
String fee = rd.readLine();
System.out.println(sid + "\t" + nm + "\t" + phn + "\t"
+ fee);
}}
```

```
class JlcReader {
public String readLine() throws IOException {
StringBuffer val = new StringBuffer();
while (true) {
int asc = System.in.read();
if (asc == 13) ;      // IGNORE 13 ASCII
else if (asc == 10)
   break;             // IGNORE 10 ASCII & BREAK
else {
char ch = (char) asc;
val.append(ch);
}
}
return val.toString();
}
}
```

- When you read the data from the keyboard then apart from the actual data when you press the ENTER then some extra ascii value will be stored in the buffer.
  - 13 For \r character (Moves the cursor to beginning of current line)
  - 10 For \n character (Moves the cursor to next line)
- When you create the string from those data then you need to ignore the extra ascii values.

Stream

- Sequence of bits is called as Stream.
- There are two type of streams based on operation:
  - Input Streams
  - Output Streams
- Input Streams are used to read the data from various input devices like keyboard, file, network etc.
- Output Streams are used to write the data to various output devices like monitor, file, network etc.
- There are two type of streams based on data:
  - Byte Stream : used to read or write byte(ASCII value) data.
  - Character Stream : used to read or write character data.

Byte Input Streams

- These Streams are used to read byte data from various input devices.
- InputStream is an abstract class and it is the superclass for all the Input Byte Streams.
- List of Byte Input Streams:

InputStream	DataInputStream
ByteArrayInputStream	FileInputStream
BufferedInputStream	ObjectInputStream

Byte Output Streams

- These Streams are used to write byte data to various output devices.
- OutputStream is an abstract class and it is the superclass for all the Output Byte Streams.
- List of Byte Output Streams:

OutputStream	DataOutputStream
ByteArrayOutputStream	FileOutputStream
BufferedOutputStream	ObjectOutputStream

Character Input Streams

- These Streams are used to read char data from various input devices.
- Reader is an abstract class and it is the superclass for all the Character Input Streams.
- List of Character Input Streams:

Reader	CharArrayReader
FileReader	InputStreamReader
BufferedReader	

Character Output Streams

- These Streams are used to write char data to various output devices.
- Writer is an abstract class and it is the superclass for all the Character Output Streams.
- List of Character Output Streams:

Writer	CharArrayWriter
FileWriter	OutputStreamWriter
BufferedWriter	PrintWriter

- There is no inheritance relation between the InputStream and the Reader class.
- If you have the InputStream object and you want to use Reader object then you must convert the InputStream into the Reader by using java.io.InputStreamReader class.
- An InputStreamReader is a bridge from byte streams to character streams.It reads bytes data and decodes into character.

- There is no inheritance relation between the OutputStream and the Writer class.
- If you have the OutputStream object and you want to use Writer object then you must convert the outputStream into the Writer by using java.io.OutputStreamWriter class.
- When you use writer object then you must invoke flush() method to send the data to the Output device.

- When you want to read the data from the buffer then you can use BufferedInputStream or BufferedReader. You need to specify the source device from where the data will be access.
  - BufferedInputStream(InputStream ins)
  - BufferedReader(Reader rd)
- When you want to read the data from the file then you can use FileInputStream or FileReader class.
  - FileInputStream(File f)
  - FileInputStream(String fname)
  - FileReader(File f)
  - FileReader(String fname)
- When you want to write the data to the file then you can use FileOutputStream or FileWriter class.
  - FileOutputStream(File f)
  - FileOutputStream(String fName)
  - FileOutputStream(File f, boolean append)
  - FileOutputStream(String fName, boolean append)
  - FileWriter(File f)
  - FileWriter(String fName)
  - FileWriter(File f, boolean append)
  - FileWriter(String fName, boolean append)

Lab1332.java

```
import java.io.*;
public class Lab1332 {
public static void main(String[] args) {
System.out.println("Enter data");
try(BufferedInputStream bis=new
BufferedInputStream(System.in);){
while(true){
int asc=bis.read();
if(asc==13) break;
char ch=(char)asc;
System.out.print(ch);
}
}catch(Exception e){
e.printStackTrace();
}
}
}
```

Lab1333.java

```
import java.io.*;
public class Lab1333 {
public static void main(String[] args) throws Exception {
try(FileInputStream fis=new
FileInputStream("D:\\D1\\abc.txt");
BufferedInputStream bis=new BufferedInputStream(fis);){
while(true){
int asc=bis.read();
if(asc==-1) break;
char ch=(char)asc;
System.out.print(ch);
}
}catch(Exception e){
e.printStackTrace();
}
}
}
```

Lab1334.java

```
import java.io.*;
public class Lab1334 {
public static void main(String[] args) {
try(
InputStreamReader isr=new
InputStreamReader(System.in);
BufferedReader br=new BufferedReader(isr);){
System.out:println("Enter Id");
String id=br.readLine();
System.out.println("Enter Name");
String nm=br.readLine();
System.out.println(id+"\t"+nm);
}catch(Exception e){
e.printStackTrace();
}
}}
```

Lab1335.java

```
import java.io.*;
public class Lab1335 {
public static void main(String[] args) {
try(
FileInputStream fis=new
FileInputStream("D:\\D1\\abc.txt");
InputStreamReader isr=new InputStreamReader(fis);
BufferedReader br=new BufferedReader(isr);){
while(true){
String id=br.readLine();
if(id==null) break;
System.out.println(id);
}
}catch(Exception e){
e.printStackTrace();
}
}
}
```

Lab1336.java

```
import java.io.*;
public class Lab1336 {
public static void main(String[] args) {
try(
FileInputStream fis=new
FileInputStream("D:\\D1\\abc.txt");
FileOutputStream fos=new
FileOutputStream("D:\\D1\\xyz.txt");){
while(true){
int asc=fis.read();
if(asc==-1)break;
fos.write(asc);
}
System.out.println("Writing Completed");
}catch(Exception e){
e.printStackTrace();
}
}}
```

Lab1337.java

```
import java.io.*;
public class Lab1337 {
public static void main(String[] args) {
try(
FileReader fr=new FileReader("D:\\D1\\abc.txt");
BufferedReader  br=new BufferedReader(fr);
FileWriter fw=new FileWriter("D:\\D1\\xyz.txt");
BufferedWriter bwr=new BufferedWriter(fw);){
while(true){
String st=br.readLine();
if(st==null)break;
bwr.write(st);
bwr.newLine();
}
bwr.close();
System.out.println("Writing Completed");
}catch(Exception e){
e.printStackTrace();
} }}
```

Lab1338.java

```
import java.io.*;
public class Lab1338 {
public static void main(String[] args) {
try(
InputStreamReader isr=new
InputStreamReader(System.in);
BufferedReader br=new BufferedReader(isr);
FileWriter fw=new
FileWriter("D:\\D1\\stinfo.txt",true);
BufferedWriter bwr=new BufferedWriter(fw);){
char ch='Y';
do{
System.out.println("Enter Id");
String id=br.readLine();
System.out.println("Enter Name");
String nm=br.readLine();
String info=id+"\t"+nm;
bwr.write(info);
bwr.newLine();
System.out.println("Do you want to add more:[Y/N]");
ch=(char)br.readLine().charAt(0);
}while(ch=='Y');
bwr.close();
}catch(Exception e){        e.printStackTrace();
} }}
```

Lab1339.java

```
import java.io.*;
public class Lab1339 {
public static void main(String[] args) throws
Exception{
PrintWriter pw= new PrintWriter("welcome.txt");
pw.println(97);
pw.write(97);
pw.println(10.0);
pw.println(true);
pw.println("JLC");
pw.close();
}
```

java.io.File class

- File is a class available in java.io package which contains various useful methods to do operations on file.

Lab1340.java

```
import java.io.*;
public class Lab1340 {
public static void main(String[] args) {
File drives[]=File.listRoots();
for(File f:drives){
long total=f.getTotalSpace();
total=total/1024/1024/1024;
long usable=f.getUsableSpace();
usable=usable/1024/1024/1024;
System.out.println(f+"\t\t\t"+total+"
GB\t\t"+usable+" GB");
}
}}
```

Lab1341.java

```
import java.io.*;
public class Lab1341 {
public static void main(String[] args) {
File file=new File("D:\\");
File files[]=file.listFiles();
for(File f:files){
System.out.println(f+" -> isFile :"+f.isFile()+" , isDir
:"+f.isDirectory());
}
}
}
```

Lab1342.java

```
import java.io.*;
public class Lab1342 {
public static void main(String[] args) {
File file=new File("D:\\D1");
File javaFiles[]=file.listFiles(new ExtFilter(".java"));
System.out.println("** JAVA FILES **");
if(javaFiles!=null)
for (File f : javaFiles)
 System.out.println(f);
else
 System.out.println("No Java File Found");
}}

class ExtFilter implements FileFilter{
String ext;
public ExtFilter(String ext) {
this.ext = ext;
}
public boolean accept(File f) {
return f.getName().endsWith(ext);
}
}
```

Lab1343.java

```
import java.io.*;
public class Lab1343 {
public static void main(String[] args) {
File file=new File("D:\\D1\\JAVA\\JLC");
System.out.println("Dir Found :"+file.exists());
file.mkdirs();
System.out.println("Dir Found :"+file.exists());
}}
```

Lab1344.java

```
import java.io.*;
public class Lab1344 {
public static void main(String[] args) throws Exception{
File file=new File("D:\\Hello.java");
System.out.println("File Found :"+file.exists());
file.createNewFile();
System.out.println("File Found :"+file.exists());
}}
```

Lab1345.java

```
import java.io.*;
public class Lab1345 {
public static void main(String[] args) {
System.out.println("File.path.separator\t:" +
File.pathSeparator);
System.out.println("File.path.separatorChar\t:"
+ File.pathSeparatorChar);
System.out.println("File.separator\t:" +
File.separator);
System.out.println("File.path.separatorChar\t:" +
File.separatorChar);
}
}
```

Lab1346.java

```
import java.io.*;
public class Lab1346 {
public static void main(String[] args) throws Exception{
File[] far = File.listRoots();
System.out.println(far.length);
System.out.println("FileName\tis
Directory\tisFile\tisAbsolute\tgetPath");
for (int i = 0; i < far.length; i++) {
System.out.println(far[i].getName() + "\t\t" +
far[i].isDirectory()+ "\t\t" + far[i].isFile() + "\t\t" +
far[i].isAbsolute()+ "\t\t" + far[i].getPath());
}
File f = new File(far[1], "hi.txt");
f.createNewFile();
}
}
```

Lab1347.java

```
import java.io.*;
public class Lab1347 {
public static void main(String[] args) {
File f=new File("d:\\hello.txt");
f.createNewFile();
f.deleteOnExit();
File f2=new File("hi.txt");
f2.createNewFile();
System.out.println("f2.isHidden()\t"+f2.isHidden());
System.out.println("f2.delete()\t"+f2.delete());
System.out.println("File Created");
File f3=new File("c:\\Documents and
Settings\\Default User");
System.out.println("f3.isHidden()\t"+f3.isHidden());
}
}
```

Lab1348.java

```
import java.io.*;
public class Lab1348 {
public static void main(String[] args) {
File f1=new File("sri");
File f2=new File(f1,"jlc\\io");
f2.mkdirs();
System.out.println("directory io inside jlc,and jlc inside sri
is created");
File f3=new File(f2,"hai.txt");
System.out.println("f3.getName()\t"+f3.getName());
f3.createNewFile();
System.out.println("file hi.txt is created sri\\jlc\\io");
}
}
```

Lab1349.java

```
import java.io.*;
public class Lab1349 {
public static void main(String[] args) throws
Exception{
File f = new File("sri\\jlc\\io");
f.mkdirs();
File f1=new File(f,"jlc.txt");
System.out.println("f1.getName()\t"+f1.getName());
System.out.println("before  creating
f1.exists()\t"+f1.exists());
f1.createNewFile();
System.out.println("file hi.txt is created sri\\jlc\\io");
System.out.println("f1.getAbsolutePath()\t:"+f1.getA
bsolutePath());
System.out.println("f1.get.Parent()\t:"+f1.getParent()
);
System.out.println("f1.get.Path()\t:"+f1.getPath());
System.out.println("after creating
f1.exists()\t"+f1.exists());
System.out.println("f1.canRead()\t"+f1.canRead());
System.out.println("f1.canWrite()\t"+f1.canWrite());
f1.setReadOnly();
System.out.println("marked read only");
System.out.println("f1.canRead()\t"+f1.canRead());
System.out.println("f1.canWrite()\t"+f1.canWrite());
File f2=new File("hello.txt");
f2.createNewFile();
System.out.println("f2.getAbsolutePath()\t:"+f2.getAbsolu
tePath());
System.out.println("f2.get.Parent()\t:"+f2.getParent());
System.out.println("f2.get.Path()\t:"+f2.getPath());}
}
```

Serialization

Serialization:

- Serialization is the process of saving state of an object into file i.e reading the data from object and writing to file.

De-serialization:

- De-serialization is the process of reading the data from file and creating object on remote machine.

Serialization can be used in the following scenarios:

1. If you want to store the data of an object in secured format so that it can be accessed by authorised Java application only.
2. By using the Serialization you can manage the memory available required for your Java application.
3. If you have the requirement to transfer the object data from one machine to another machine through the network.

- With RMI(remote method invocation) or EJB(Enterprise Java Beans) you need to invoke the method which is running on the remote machine.
- When you are invoking any method then you need to pass either primitive or refence type of data.
- When you are invoking remote method by passing primitive type then directly value will copied to remote machine.

- When you are invoking remote method by passing reference type then address of an object will be copied to remote machine. But there is no use of passing address of one JVM to another JVM.
- So to pass the object from one machine to another machine you have to save the state of an object to the file.
- State of the object means the current value of the instance member.
- You can use the following classes for this purpose:
  - ObjectOutputStream
  - ObjectInputStream
- To save the state of an Object to file you can use void writeObject(Object) method of ObjectOutputStream class.
- To create the object from file on remote machine you can use Object readObject() method of ObjectInputStream class.
- By default any objects are not eligible for serialization so if you want to serialize any class object then that class has to implement java.io.Serializable marker interface.
- If you are trying to serialize the object without implementing Serializable interface then following exception will be thrown: java.io.NotSerializableException.
- serialVersionUID: This specifies your class version information while serializing and the same will be used at the time of deserialization.
- When you are deserialising the object then:
  - The static variable will get the value defined in the class(because static member will not be serialized with the object).
  - Same .class file should be available that was used at the time of Serailization.
  - The constructor of your class will not be used.

Lab1350.java

```
import java.io.*;
public class Lab1350 {
public static void main(String[] args) throws
Exception{
try(FileOutputStream fos=new
FileOutputStream("D:\\info.ser");
ObjectOutputStream oos = new
ObjectOutputStream(fos);){
Student st = new Student(99,"Sri",65799999);
Student.count=9;
System.out.println(st);
oos.writeObject(st);
System.out.println("Object Serialized");
}catch(Exception e){
 e.printStackTrace();
}
}}
```

```
class Student implements Serializable{
int sid;
String name;
long phone;
static int count=3;
Student(int sid,String name,long phone){
this.sid=sid;
this.name = name;
this.phone = phone;
}
public String toString() {
return sid+"\t"+name+"\t"+phone+"\t"+count;
}
}
```

- When you are invoking remote method by passing reference type then address of an object will be copied to remote machine. But there is no use of passing address of one JVM to another JVM.
- So to pass the object from one machine to another machine you have to save the state of an object to the file.
- State of the object means the current value of the instance member.
- You can use the following classes for this purpose:
  - ObjectOutputStream
  - ObjectInputStream
- To save the state of an Object to file you can use void writeObject(Object) method of ObjectOutputStream class.
- To create the object from file on remote machine you can use Object readObject() method of ObjectInputStream class.
- By default any objects are not eligible for serialization so if you want to serialize any class object then that class has to implement java.io.Serializable marker interface.
- If you are trying to serialize the object without implementing Serializable interface then following exception will be thrown: java.io.NotSerializableException.
- serialVersionUID: This specifies your class version information while serializing and the same will be used at the time of deserialization.
- When you are deserialising the object then:
  - The static variable will get the value defined in the class(because static member will not be serialized with the object).
  - Same .class file should be available that was used at the time of Serailization.
  - The constructor of your class will not be used.

Lab1350.java

```
import java.io.*;
public class Lab1350 {
public static void main(String[] args) throws
Exception{
try(FileOutputStream fos=new
FileOutputStream("D:\\info.ser");
ObjectOutputStream oos = new
ObjectOutputStream(fos);){
Student st = new Student(99,"Sri",65799999);
Student.count=9;
System.out.println(st);
oos.writeObject(st);
System.out.println("Object Serialized");
}catch(Exception e){
 e.printStackTrace();
}
}}
```

```
class Student implements Serializable{
int sid;
String name;
long phone;
static int count=3;
Student(int sid,String name,long phone){
this.sid=sid;
this.name = name;
this.phone = phone;
}
public String toString() {
return sid+"\t"+name+"\t"+phone+"\t"+count;
}
}
```

De-Serialized data of pre-program

Lab1351.java

```
import java.io.*;
public class Lab1351 {
public static void main(String[] args) throws
Exception{
try(FileInputStream fis=new
FileInputStream("D:\\info.ser");
ObjectInputStream ois = new
ObjectInputStream(fis);){
Object obj=ois.readObject();
System.out.println(obj);
System.out.println("Object Deserialized");
}catch(Exception e){
        e.printStackTrace();
}
}
}
// Student.class file is required created in the Last example
```

read the object

- If you want you can serialize the individual property of the object or the class explicitly also.
- To serialize the individual property you need to use the corresponding writeX() method from the ObjectOutputStream. You can read these properties by using the methods from the ObjectInputStream depending on datatype.
- In which order you are serializing the data in the same order you need to de-serialize.
- If you are serializing some object that has property inherited from the superclass then
  - If the superclass is implementing java.io.Serializable then those inherited property will be serialized.
  - If the superclass is not implementing java.io.Serializable then the inherited property will not be serialized.
- At the time of deserializing the object,
  - The property inherited from the non-serializable superclass will get the value defined in the class.
  - Default constructor of the non-serializable super class will be invoked.(If not available then exception will be thrown).

Lab1352.java

```
import java.io.*;
public class Lab1352 {
public static void main(String[] args) throws
Exception{
try(FileOutputStream fos=new
FileOutputStream("D:\\info.ser");
ObjectOutputStream oos = new
ObjectOutputStream(fos);){
Student st = new Student(99,"Sri",65799999);
Student.count=9;
System.out.println(st);
oos.writeObject(st);
System.out.println("Object Serialized");
}catch(Exception e){
        e.printStackTrace();
}
}}
```

```
// class Person implements Serializable {
class Person{
String name="XXXX";
long phone=333333;
}
class Student extends Person implements Serializable{
int sid;
static int count=3;
Student(int sid,String name,long phone){
this.sid=sid;       this.name = name;  this.phone = phone;
}
public String toString() {
return sid+"\t"+name+"\t"+phone+"\t"+count;
}}

// Use Lab1351 code to deserialize the object
// Student and Person class will be from Lab1352
```

transient keyword

- It is a keyword in Java that will be used with the variable of the class. This keyword is useful in the serialization process. If you are serializing the object then by default all the instance variables will be serialized.
- If you want to restrict that some variables should not be serialized then those variables can be defined as transient. While deserializing the object these variables will get the default value depending on the datatype.
- If the object you want to serialize has the serializable instance member object then member object also will serialized.
- If you don't want to serialize the member object then define as transient.

Lab1353.java

```
import java.io.*;
public class Lab1353 {
public static void main(String[] args) throws
Exception{
try(FileOutputStream fos=new
FileOutputStream("D:\\info.ser");
ObjectOutputStream oos = new
ObjectOutputStream(fos);){
Student st = new Student(99,"Sri",65799999);
Student.count=9;
System.out.println(st);
oos.writeObject(st);
System.out.println("Object Serialized");
}catch(Exception e){
        e.printStackTrace();
} }}
```

```
class Student implements Serializable{
int sid;
transient String name="XXXX";
long phone=333333;
static int count=3;
Student(int sid,String name,long phone){
this.sid=sid;
this.name = name;
this.phone = phone;
}
public String toString() {
return sid+"\t"+name+"\t"+phone+"\t"+count;
}
}

// Use Lab1351 code to deserialize the object
// Student and Person class will be from Lab1353
```

Lab1354.java

```
import java.io.*;
public class Lab1354 {
public static void main(String[] args) throws
Exception{
try(FileOutputStream fos=new
FileOutputStream("D:\\info.ser");
ObjectOutputStream oos = new
ObjectOutputStream(fos);){
        Address ad=new
Address("33/1","Mathikere");
Student st = new Student(99,"Sri",65799999,ad);
Student.count=9;
System.out.println(st);
oos.writeObject(st);
```

```
class Student implements Serializable{
int sid;
transient String name="XXXX";
long phone=333333;
static int count=3;
Address add;
Student(int sid,String name,long phone,Address add){
this.sid=sid;
this.name = name;
this.phone = phone;
this.add=add;
}
public String toString() {
return
```

```
System.out.println("Object Serialized");
}catch(Exception e){
        e.printStackTrace();
}
}
}

// class Address implements Serializable {
class Address{
String aid;
String street;
Address(String aid,String street){
this.aid = aid;
this.street = street;
}
public String toString() {
return aid+"\t"+street;
}
}
```

```
sid+"\t"+name+"\t"+phone+"\t\t"+count+"\n"+add;
}
}

// Use Lab1351 code to deserialize the object
// Student and Person class will be from Lab1354
```

Externalization

- Java Vendor has provided some default implementation for serialization.
- Externalization is the process of customizing serialization process.
- By using Externalization concept you can serialize particular fileds of your class.
- To perform Externalization, your class has to implement java.io.Externalizable interface.
- This is not a marker interface like java.io.Serializable. This contains following two abstract methods:-
  - void readExternal(ObjectInput);
  - void writeExternal(ObjectOutput);
- writeExternal() method will take ObjectOutput stream as a parameter and inside this method you have to specify which fields of your class you want to serialize.
- readExternal() method will take ObjectInput stream as a parameter and inside this method you have to deserialze particular field of your class.
- writeExternal() will be called while serializing the object.
- readExternal() will be called while deserializing the object.
- Your class must be have public no-argument constructor.

Lab1355.java

```
import java.io.*;
public class Lab1355 {
public static void main(String[] args) throws
Exception{
try(FileOutputStream fos=new
FileOutputStream("D:\\info.ser");
ObjectOutputStream oos = new
ObjectOutputStream(fos);){
        Address ad=new
Address("33/1","Mathikere");
Student st = new Student(99,"Sri",65799999,ad);
Student.count=9;
System.out.println(st);
oos.writeObject(st);
System.out.println("Object Serialized+");
}catch(Exception e){
        e.printStackTrace();
}
}
}

class Address{
String aid;
String street;
Address(){}
Address(String aid,String street){
this.aid = aid;
this.street = street;
}
public String toString() {
return aid+"\t"+street;
}
}
```

```
class Student implements Externalizable{
int sid;
transient String name="XXXX";
long phone=333333;
static int count=3;
Address add;
public Student(){}
Student(int sid,String name,long phone,Address add){
this.sid=sid;
this.name = name;
this.phone = phone;
this.add=add;
}
public String toString() {
return
sid+"\t"+name+"\t"+phone+"\t\t"+count+"\n"+add;
}
public void writeExternal(ObjectOutput out) throws
IOException {
        out.writeInt(count);
        out.writeObject(name);
        out.writeObject(add.aid);
        out.writeObject(add.street);
}
public void readExternal(ObjectInput in) throws
IOException,
                ClassNotFoundException {
        count = in.readInt();
        name = in.readObject().toString();
        add=new Address();
        add.aid = in.readObject().toString();
        add.street=in.readObject().toString();
}
}

// Use Lab1351 code to deserialize the object
// Student and Person class will be from Lab1355
```